Лабораторная работа №4 «Решение задач оптимизации с использованием пакета DEAP».

1. Решение задач поиска минимальных маршрутов в графе.

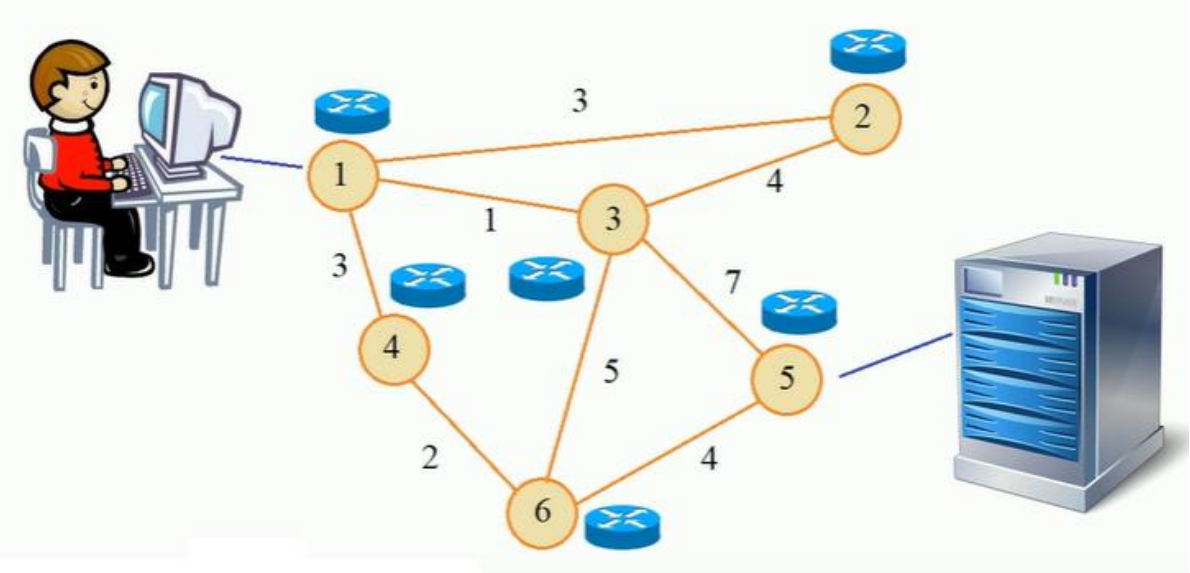
Задача поиска минимальных маршрутов в графе.

Такая задача может быть решена с использованием алгоритма Дейкстры. Суть алгоритма – определение наикратчайших маршрутов от некоторой стартовой вершины графа (т.е. найти маршрут с минимальными суммарными связями).

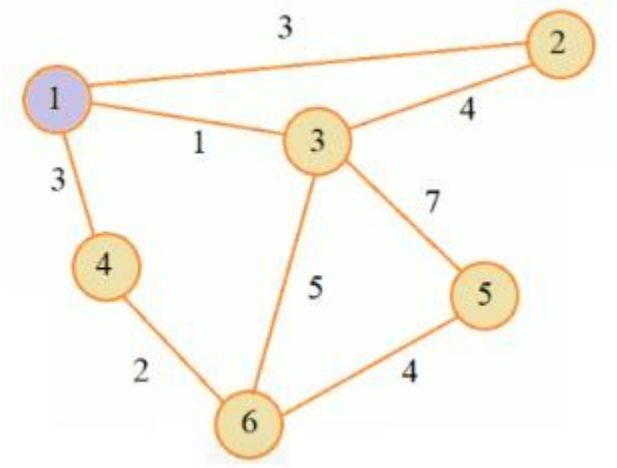
Исходные данные - граф с дугами, которые имеют положительные веса. Пример использования таких графов – это, например, задача определения оптимального маршрута (клиент (узел 1) отправляет запрос серверу (узел 5)). Веса дуг могут обозначать среднее время передачи пакета.

Таким образом, с использованием алгоритма Дейкстры, можно найти маршрут с минимальным временем передачи информации.

На практики, таких задач может быть много.



Решим задачу с использованием ГА.



Исходный граф

При использовании генитического алгоритма необходимо определится со способами хранения информации в хромосомах. Здесь могут быть самые разные варианты. Исходная точка **вершина №1 (узел 0**).

Одна хромасома описывает все возможные маршруты от исходной точки (т.е. от узла **0** в нашем случае это будет **вершина №1**) до последнего (т.е. до **5** в нашем случае это будет вершина **№6**).

При этом каждый список в хромосоме определяет кратчайший маршрут из **вершины (узла) 0** ко всем остальным вершина (узлам), т.е. кратчайший маршрут из вершины **0 в** вершину **0** это **[0, 3, 1, 5, 2, 4], из 0 в 1** это **[1, 0, 4, 3, 5, 2]** и т.д.

Узлы (вершины):

0	1	2	3	4	5

[[0, 3, 1, 5, 2, 4], [1, 0, 4, 3, 5, 2], [2, 4, 1, 5, 3, 0], [0, 3, 5, 4, 2, 1], [2, 4, 1, 0, 5, 3], [3, 5, 0, 4, 1, 2]]

Маршруты:

из 0 в 0	из 0 в 1	из 0 в 2	из 0 в 3	из 0 в 4	из 0 в 5

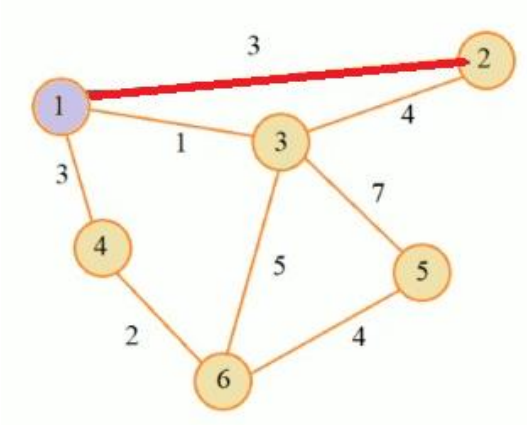
Так как мы не знаем какой этот кратчайший маршрут, и сколько вершин будет включено в этот кратчайший маршрут, то мы в хромосоме храним список из всех возможных вершин. Заканчиваться маршрут будет тогда, когда встретится целевая вершина.

Например, двигаемся к вершине 0, то целевая вершина 0 и здесь маршрут заканчивается. Кратчайший путь будет таже самая точка, т.е. двигаться ненужно.

```
Узлы (вершины):
      0                1
[[0, 3, 1, 5, 2, 4],  [1, 0, 4, 3, 5, 2],
Маршруты:
    из 0 в 0        из  0 в 1
```

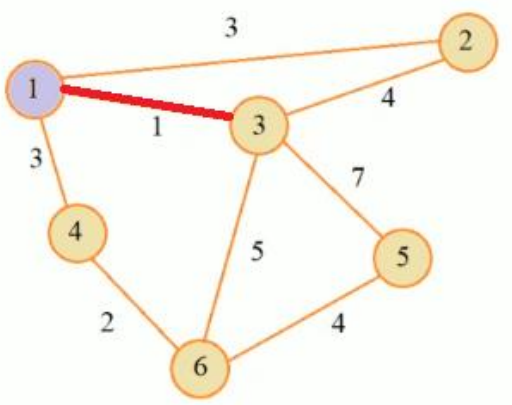
Например, двигаемся к вершине 1 из 0, то целевая вершина 1. Кратчайший маршрут из вершины 0 в 1 – это ребро, которое соединяет это две вершины.

```
Узлы (вершины):
      0                1
[[0, 3, 1, 5, 2, 4],  [1, 0, 4, 3, 5, 2], [2,
Маршруты:
    из 0 в 0        из  0 в 1
```

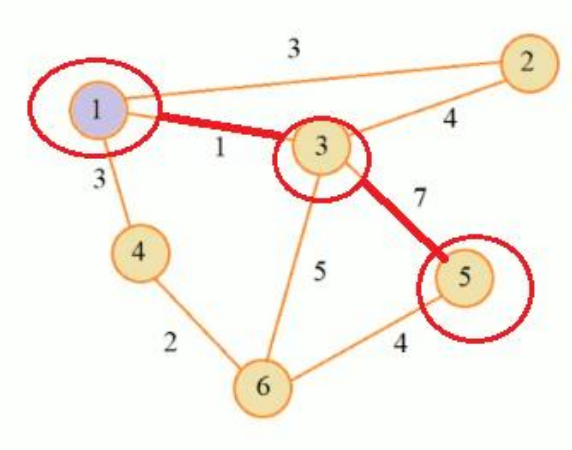


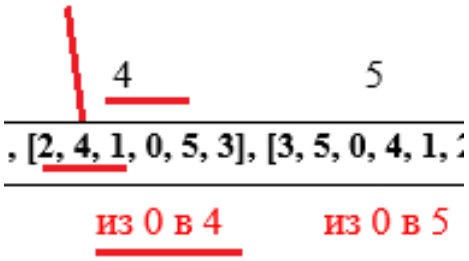
Кратчайший маршрут из вершины 0 в 2 – это тоже прямой путь (ребро, которое соединяет это две вершины)

```
Узлы (вершины):
      0                1                2
[[0, 3, 1, 5, 2, 4],  [1, 0, 4, 3, 5, 2], [2, 4, 1, 5, 3, 0], [
Маршруты:
    из 0 в 0        из  0 в 1        из 0 в 2
```

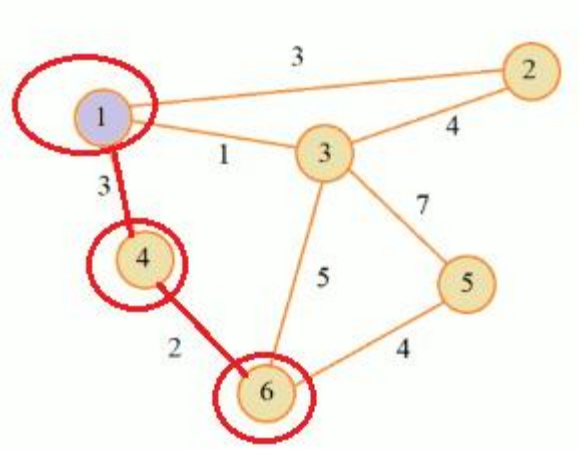


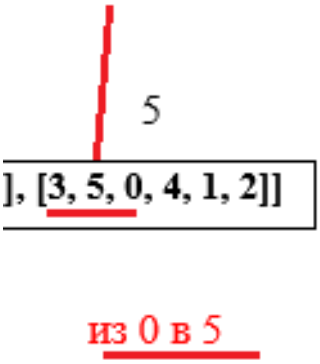
Путь узла 0 в 4 означает, что мы выходим из вершины №1, заходим в вершину №3, а затем только в вершину №5 (узел 4)





Если необходимо попасть в вершину №6, то маршрут будет.



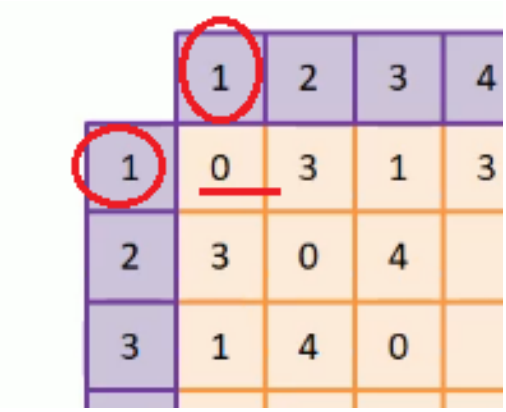


Это был один из способов кодирования информации в хромосоме. Можно было предложить другой способ.

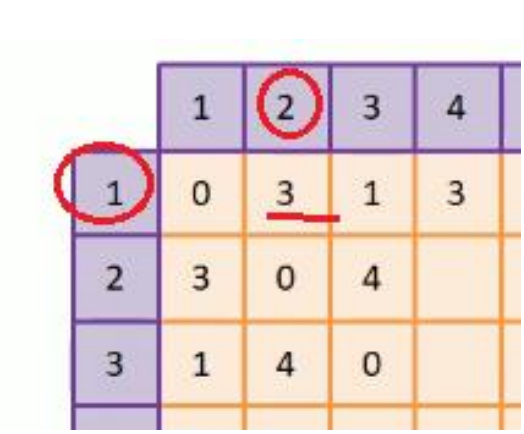
Следующий важный момент – это как хранить информацию о графе в прграмме. Обычно для этого используется матрица смежности. Здесь по строкам и по столбцам отложены номера вершин.

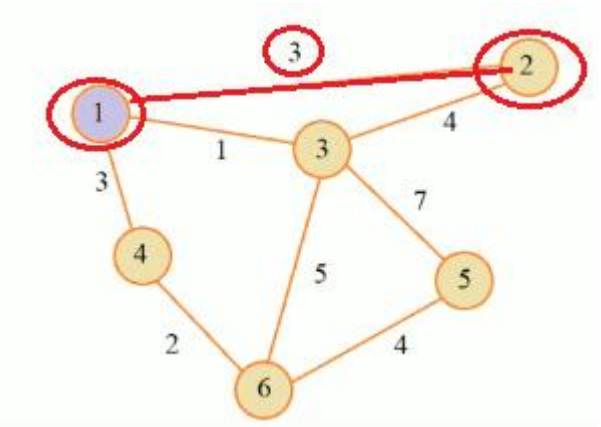
	1	2	3	4	5	6
1	0	3	1	3		
2	3	0	4			
3	1	4	0		7	5
4	3			0		2
5			7		0	4
6			5	2	4	0

Если идем из вершины №1 в вершину №1, то длина маршрута равна 0.



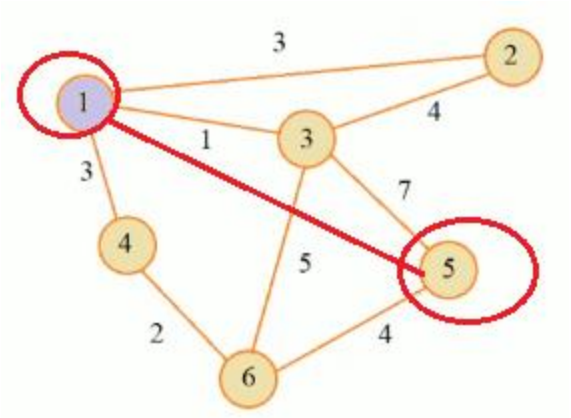
Если идем из вершины №1 в вершину №2, то длина маршрута равна 3.





Если вершины на прямую не соединины, то эта клетка пустая, например вершина №1 и вершина №5. Предполагается, что это бесконечно большое число.

Так будет представлен граф в нашей программе.



	1	2	3	4	5	6
1	0	3	1	3	0	
2	3	0	4			
3	1	4	0		7	5

Для реализаци ГА импортируем необходимые библиотеки.

```
course > ex1.py
ex1.py
1  from deap import base, algorithms
2  from deap import creator
3  from deap import tools
4
5  import random
6  import matplotlib.pyplot as plt
7  import numpy as np
8
```

Далее зададим матрицу смежности. Пустым клеткам присваиваем значение 100 (inf=100)

```
8
9   inf = 100
10  D = ((0, 3, 1, 3, inf, inf),
11       (3, 0, 4, inf, inf, inf),
12       (1, 4, 0, inf, 7, 5),
13       (3, inf, inf, 0, inf, 2),
14       (inf, inf, 7, inf, 0, 4),
15       (inf, inf, 5, 2, 4, 0))
```

Когда ГА будет включать не существующие дуги графа, то это сразу даст прибавку в 100 единиц. Такой индивидуум будет проигровать тем, которые

используют толька допустимые дуги графа. Поэтому ГА будет старатся использовать только допустимым маршруты между узлами.

Далее определим необходимые константы. Во первых это будет номер вершины от которой мы стартуем (**startV 0 – это первая вершина**). Общее число вершин в графе (**LENGTH_D это количество строк в матрице D**). А общее количество элементов в хромосоме определяем **len(D)*len(D[0])**. В данном случае их будет 6 на 6 т.е. 36.

```
8
9   inf = 100
10  D = ((0, 3, 1, 3, inf, inf),
11       (3, 0, 4, inf, inf, inf),
12       (1, 4, 0, inf, 7, 5),
13       (3, inf, inf, 0, inf, 2),
14       (inf, inf, 7, inf, 0, 4),
15       (inf, inf, 5, 2, 4, 0))
16
17  startV = 0              # стартовая вершина
18  LENGTH_D = len(D)
19  LENGTH_CHROM = len(D)*len(D[0])    # длина хромосомы, подлежащей оптимизации
20
```

Далее, пропишим параметры ГА.

```
17  startV = 0              # стартовая вершина
18  LENGTH_D = len(D)
19  LENGTH_CHROM = len(D)*len(D[0])    # длина хромосомы, подлежащей оптимизации
20
21  # константы генетического алгоритма
22  POPULATION_SIZE = 500   # количество индивидуумов в популяции
23  P_CROSSOVER = 0.9       # вероятность скрещивания
24  P_MUTATION = 0.1        # вероятность мутации индивидуума
25  MAX_GENERATIONS = 30    # максимальное количество поколений
26
```

Далее, для повторяемости результатов экспериментов работы ГА определим зерно для генератора случайных чисел (**RANDOM_SEED=42**)

```
25  MAX_GENERATIONS = 30    # максимальное количество поколений
26
27  RANDOM_SEED = 42
28  random.seed(RANDOM_SEED)
29
```

Далее, в программе создадим класс, который будет называться “**FitnessMin**” для хранении информации приспособленности каждого индивидуума. Класс “**Individual**”, будет наследоваться от класса “**list**” (т.е. список) для представления отдельной особи в популяции. Обратите внимания, в “**FitnessMin**” берем вес равный -1.0. Знак «-», т.к. будем выполнять минимизацию функции приспособленности каждой отдельной особи (т.е. будем искать кратчайший маршрут).

```
29
30  creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
31  creator.create("Individual", list, fitness=creator.FitnessMin)
32
```

Далее, создадим начальную популяцию особей в ГА. Для этого мы воспользуемся известным классом “**Toolbox**” и через его экземпляр, т.е. через переменные“**toolbox**” выполним регистрацию нужных нам функций. Это будет функция “**randomOrder**”, которая на выходе дает (**random.sample**) список из не повторяющихся числовых значений (т.е. **sample** будет давать случайные списки, со случайными значениями, но в пределах списка значения будут меняться от 0 до 5 и не повторяться).

Узлы (вершины):

0 1 2

[[0, 3, 1, 5, 2, 4], [1, 0, 4, 3, 5, 2], [2, 4, 1, 5, 3, 0],

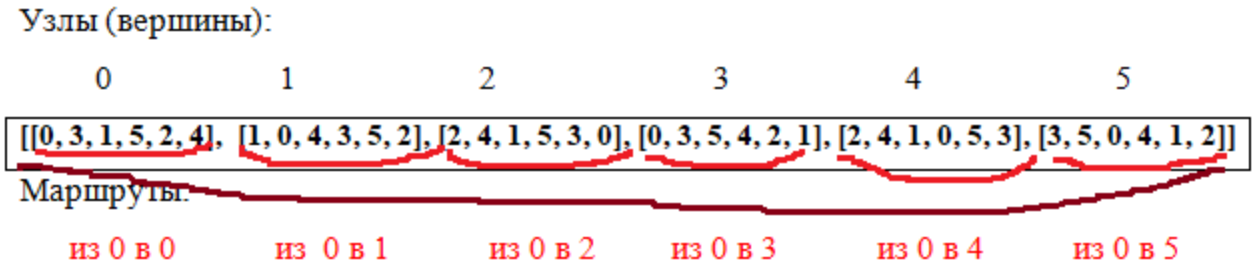
Маршруты:

из 0 в 0 из 0 в 1 из 0 в 2 и т.д.

```
30  creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
31  creator.create("Individual", list, fitness=creator.FitnessMin)
32
33  toolbox = base.Toolbox()
34  toolbox.register("randomOrder", random.sample, range(LENGTH_D), LENGTH_D)
35  toolbox.register("individualCreator", tools.initRepeat, creator.Individual, toolbox.randomOrder, LENGTH_D)
36  toolbox.register("populationCreator", tools.initRepeat, list, toolbox.individualCreator)
37
```

Далее функция "**individualCreate**" будет создавать отдельного индивидуума в нашей программе. Для этого будем использовать класс creator.**Individual (**"**Individual**") . Он будет хранить все списки. Каждый элемент этого списка будет формироваться с помощью функции **randomOrder** (toolbox.**randomOrder**), а количество элементов этого списка будет определяться **LENGTH_D,** т.е. будет 6.

В итого этот "**individualCreate**" нам создаст общий список, состоящий их 6 вложенных списков.



Последняя функция "**populationCreator**" создает популяцию из индивидуумов (**individualCreate**).

Далее, вызовем эту функцию и создадим популяцию размером **POHULATION_SIZE**.

```
toolbox.register("populationCreator", tools.initRepeat, list, toolbox.individualCreator)

population = toolbox.populationCreator(n=POPULATION_SIZE)
```

Далее, определим вычисление приспособленности отдельных особей. Для этого создадим функцию (**dikstryFitness**), которая должно использовать матрицу смежности и номер узла(вершины) от которого рассчитываются длины маршрутов до всех остальных вершин графа. Такую функцию реализуем следующим образом.

```
population = toolbox.populationCreator(n=POPULATION_SIZE)

def dikstryFitness(individual):
    s = 0
    for n, path in enumerate(individual):
        path = path[:path.index(n)+1]

        si = startV
        for j in path:
            s += D[si][j]
            si = j

    return s,          # кортеж
```

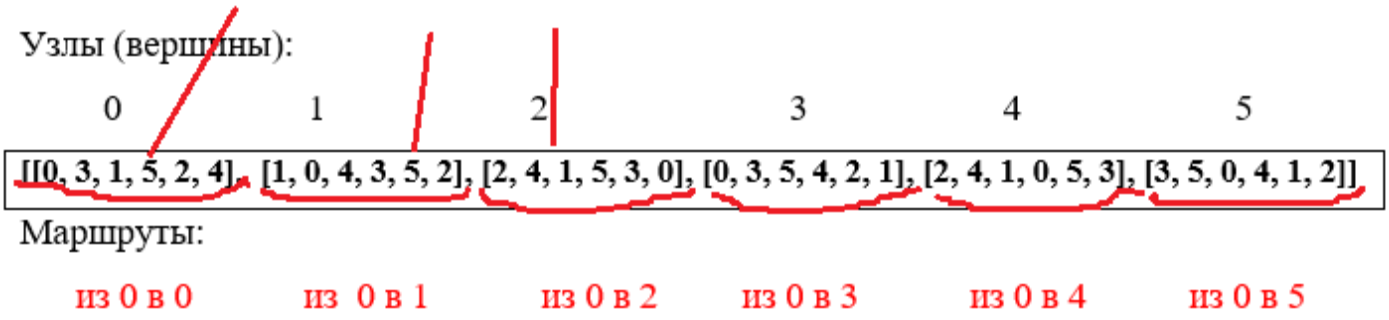
Переменная **S** хранит суммарный маршрут до всех вершин графа. Далее перебирая каждый список в хромосоме индивидуума (**individual**) и операции среза выделяем из списка только те узлы (**path[:path.index(x)+1]**), которые относятся к текущему маршруту. Метод **index** у списка возвращает индекс элемента со значением **n**, а значение **n** – это текущее номер узла до которого мы определяем маршрут. Таким образом, операция **path.index(x)+1** возвратит маршрут до текущего узла.

Далее, во втором вложенном цикле мы непосредственно подсчитываем длину текущего маршрута используя значение матрицы смежности **D**. В конце функция **retrun** возвращает суммарный маршрут в виде кортежа. Так работает функция приспособленности **dikstryFitness**, т.е. она вычисляет суммарные маршруты до всех вершин графа.

Следующая функция **cxOrdered** определяет способ скрещивания двух особей **ind1**, **ind2**. Она работает очень просто. В цикле мы перебираем пары маршрутов (**p1, p2…**) из первой и второй особи, а затем вызываем встроенную функцию **cxOrdered** пакета DEAP, которая реализует алгоритм упорядоченного скрещивания.

```
58      return s,           # кортеж
51
52  def cxOrdered(ind1, ind2):
53      for p1, p2 in zip(ind1, ind2):
54          tools.cxOrdered(p1, p2)
55
56      return ind1, ind2
57
```

Данный способ скрещивания гарантирует сохранение всех индексов в хромосоме, т.е. номера графов, которые здесь фигурируют.



Они после скрещивания здесь дублироваться не будут, т.е. останутся уникальные индексы в этих списках. Поэтому при решении данной задачи мы используем функцию **cxOrdered** для скрещивания.

Следующая функция будет **mutShuffleIndexes** определяет порядок мутации генов в хромосоме. Здесь мы так же перебираем списки маршрутов и для каждого из них выполняем перемешивания индексов с вероятностью **indpb**. Здесь мы тоже используем встроенную функцию **(tools.mutShuffleIndexes (ind, indpb))** пакета DEAP, которая выполнят перемешивание индексов в списке.

Таким образом, мутация – **mutShuffleIndexes,** скрещивание – **cxOrdered,** вычисляем приспособленность особей – **dikstryFitness.**

```
55
56      return ind1, ind2
57
58  def mutShuffleIndexes(individual, indpb):
59      for ind in individual:
60          tools.mutShuffleIndexes(ind, indpb)
61
62      return individual,
63
```

Далее зарегистрируем необходимые функции для реализации ГА.

```
62      return individual,
63
64  toolbox.register("evaluate", dikstryFitness)
65  toolbox.register("select", tools.selTournament, tournsize=3)
66  toolbox.register("mate", cxOrdered)
67  toolbox.register("mutate", mutShuffleIndexes, indpb=1.0/LENGTH_CHROM/10)
68
```

Функция **evaluate**, которая рассчитывает непосредственно приспособленность особей, будет опираться на функцию **dikstryFitnes.**

Функция отбора **select** будет реализовывать турнирный отбор (**selTournament**).

Функция скрещивания **mate** будет использовать функцию **cxOrdered.**

Функция мутация **mutat**e будет использовать функцию **mutShuffleIndexes.** Причем вероятность перемешивания индексов в списки определяется величиной **indpb=1.0/LENGTH_CHROM/10** (т.е. длина хромосомы деленная на 10).

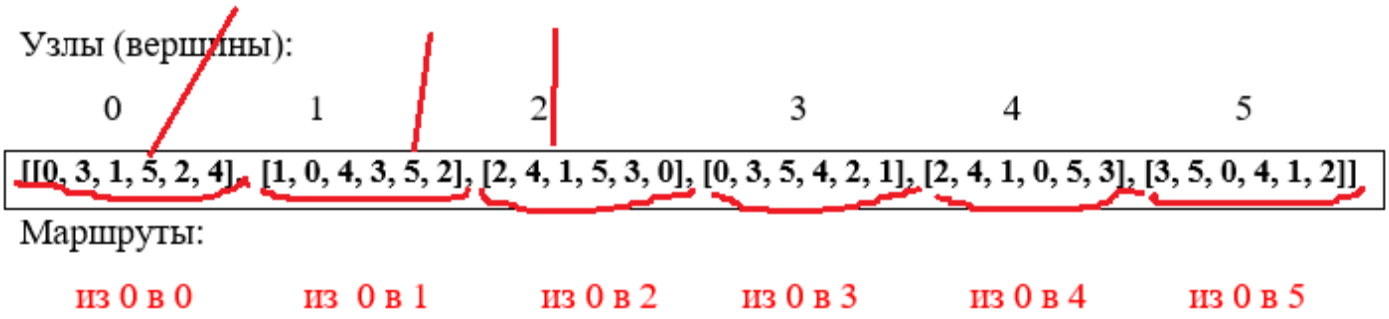
Для сбора статистики определим еще один объект **Statistic**s, который будет собирать статистику по значениям приспособленности индивидуумов в популяции на каждой итерации работы ГА. Будем определять минимальное значение (**min**) и среднее значение (**mean**) приспособленности всех особей в популяции.

```
67  toolbox.register("mutate", mutShuffleIndexes, indpb=1.0/LENGTH_CHROM/10)
68
69  stats = tools.Statistics(lambda ind: ind.fitness.values)
70  stats.register("min", np.min)
71  stats.register("avg", np.mean)
72
```

Далее, для запуска ГА воспользуемся стандартной функцией **ea.Simple** и передадим необходимые параметры. Вероятность скрещивания определена как **cxpb=P_CROSSOVER/LENGTH_D**, т.е. **cxpb=0.9/количество списков в хромосоме**, т.е. чтобы вероятность **P_CROSSOVER** была соотнесена с каждым списком.

```
71  stats.register("avg", np.mean)
72
73  population, logbook = algorithms.eaSimple(population, toolbox,
74                                             cxpb=P_CROSSOVER/LENGTH_D,
75                                             mutpb=P_MUTATION/LENGTH_D,
76                                             ngen=MAX_GENERATIONS,
77                                             stats=stats,
78                                             verbose=True)
79
```

Вероятность **0,9** относилась бы к каждому списку.



Аналогично и для вероятности мутации **mutpb**.

Далее, указываем, что ГА будет выполняться 30 раз (**MAX_GENERATION**=30). Указываем, что собираем статистику с помощью объекта **stats**, включен режим **verbose**. На каждой итерации работы алгоритма будем видеть служебную информацию в консоли.

После того, как алгоритм отработает, выберем минимальные значения, средние значения и отобразим их в виде графика.

Теперь можно запустить программу.

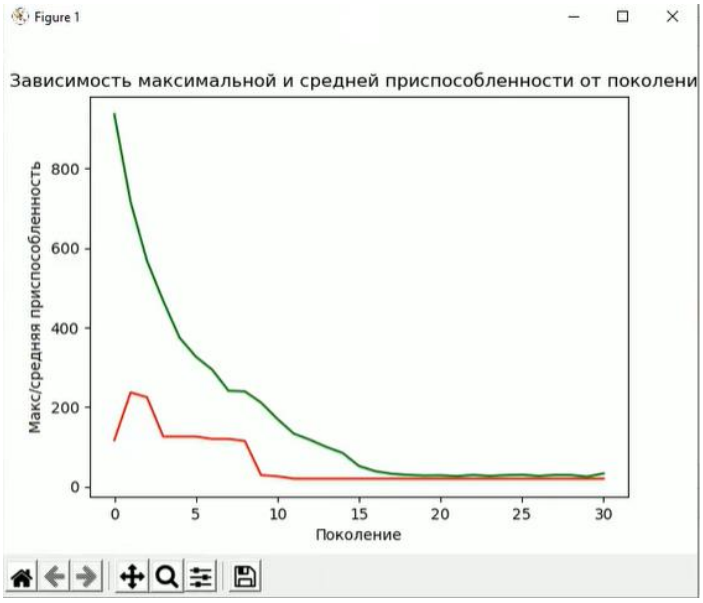
```
                                        verbose=True)

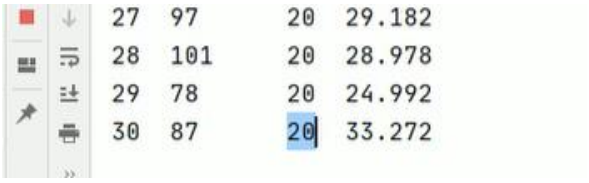
maxFitnessValues, meanFitnessValues = logbook.select("min", "avg")

plt.plot(maxFitnessValues, color='red')
plt.plot(meanFitnessValues, color='green')
plt.xlabel('Поколение')
plt.ylabel('Макс/средняя приспособленность')
plt.title('Зависимость максимальной и средней приспособленности от поколения')

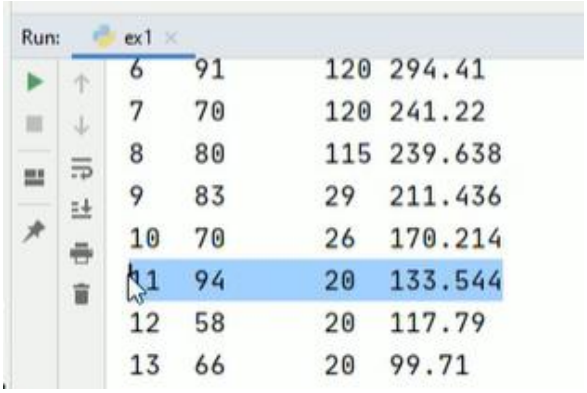
plt.show()
```

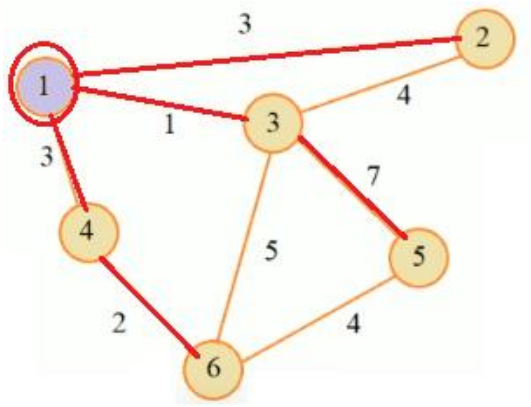
Программа отработала.





Минимальное значение равно 20, т.е. суммарный минимальный маршрут равен 20 для данного графа. Из графика видно (красный график), что этого значения достигли на 11…12 итерации.





Проверим полученные данные: из 1 в 2 вес 3; из 1 в 3 вес 1; из 1 в 5 вес 1+7; 1 в 4 вес 3; из 1 в 6 вес 2. Общий вес 3 + 1 + (1 + 7) + 3 + (3 + 2) = 20.

Таким образом, достаточно было 11 поколений (итераций), чтобы найти наилучшее, оптимальное решение поставленной задачи.

Добавим в генетический алгоритм «Зал славы», который на каждой итерации будет хранить лучших индивидуумов в популяции. Для этого пропишем еще один параметр **HALL_OF_FAME_SIZE=1**, т.е. **1** это размер «Зала славы». Далее необходимо создать специальный объект **HallOfFame** и определяется размер «Зала славы» (одна особь).

hof=tools.HallOfFame(HALL_OF_FAME_SIZE).

```
24    P_MUTATION = 0.1         # вероятность мутации индивидуума
25    MAX_GENERATIONS = 30     # максимальное количество поколен
26    HALL_OF_FAME_SIZE = 1
27
28    hof = tools.HallOfFame(HALL_OF_FAME_SIZE)
29
30    RANDOM_SEED = 42
```

Затем объект **hof** экземпляр класса **HallOfFame** должны передать в качестве параметра функции **ea.Simple** (**halloffame=hof**).

```
75
76  population, logbook = algorithms.eaSimple(population, toolbox,
77                                            cxpb=P_CROSSOVER/LENGTH_D,
78                                            mutpb=P_MUTATION/LENGTH_D,
79                                            ngen=MAX_GENERATIONS,
80                                            stats=stats,
81                                            halloffame=hof,
82                                            verbose=True)
```

Теперь можно увидеть лучшего индивидуума. Для этого обращаемся к объекту **hof,** у него есть коллекция **items** и выбираем первый элемент [0]. Это будет лучший представитель из «Зала славы» и распечатываем его в консоли.

```
83
84  maxFitnessValues, meanFitnessValues = logbook.select("min", "avg")
85
86  best = hof.items[0]
87  print(best)
88
```

Запускаем программу, выводятся данные и графики, но кроме этого видим лучшую хромосому, которая является решением поставленной задачи.

```
30  87      20  33.272
[[0, 1, 4, 3, 5, 2], [1, 4, 3, 0, 5, 2], [2, 0, 5, 1, 4, 3], [3, 2, 4, 5, 1, 0], [2, 4, 1, 5, 3, 0], [3, 5, 1, 4, 0, 2]
```

Таким образом, из представленного решения видно, что наикратчайший маршрут из точки 0 в точку 0 – это 0 [[0, 1, 4, 3, 5, 2], , т.е. идти не надо. Наикратчайший маршрут из точки 0 в точку 1 – это 1 [1, 4, 3, 0, 5, 2], . Наикратчайший маршрут из точки 0 в точку 2 – это 2, т.е. прямой путь [2, 0, 5, 1, 4, 3], Наикратчайший маршрут из точки 0 в точку 3 – это 3 [3, 2, 4, 5, 1, 0], . Наикратчайший маршрут из точки 0 в точку 4, нужно пройти через промежуточную точку 2 [2, 4, 1, 5, 3, 0], . Наикратчайший маршрут из точки 0 в точку 5, нужно пройти через промежуточную точку 3 [3, 5, 1, 4, 0, 2] .

Для представления информации в виде графа создадим дополнительный файл **graph_show.py**, который находится в одном каталоги с предыдущей программой, которая реализует ГА.

В файле **graph_show.py** создана функция **show_ graph**, которая отображает граф с лучшими маршрутами.

Отображаем все связи между вершинами графа.

ex1.py graph_show.py

```python
from matplotlib.lines import Line2D
from matplotlib.patches import Rectangle
import numpy as np

vertex = ((0, 1), (1, 1), (0.5, 0.8), (0.1, 0.5), (0.8, 0.2), (0.4, 0))

vx = [v[0] for v in vertex]
vy = [v[1] for v in vertex]

def show_graph(ax, best):
    ax.add_line(Line2D((vertex[0][0], vertex[1][0]), (vertex[0][1], vertex[1][1]), color='#aaa'))
    ax.add_line(Line2D((vertex[0][0], vertex[2][0]), (vertex[0][1], vertex[2][1]), color='#aaa'))
    ax.add_line(Line2D((vertex[0][0], vertex[3][0]), (vertex[0][1], vertex[3][1]), color='#aaa'))
    ax.add_line(Line2D((vertex[1][0], vertex[2][0]), (vertex[1][1], vertex[2][1]), color='#aaa'))
    ax.add_line(Line2D((vertex[2][0], vertex[5][0]), (vertex[2][1], vertex[5][1]), color='#aaa'))
    ax.add_line(Line2D((vertex[2][0], vertex[4][0]), (vertex[2][1], vertex[4][1]), color='#aaa'))
    ax.add_line(Line2D((vertex[3][0], vertex[5][0]), (vertex[3][1], vertex[5][1]), color='#aaa'))
    ax.add_line(Line2D((vertex[4][0], vertex[5][0]), (vertex[4][1], vertex[5][1]), color='#aaa'))
```

Координаты вершин заданы выше.

```python
import numpy as np

vertex = ((0, 1), (1, 1), (0.5, 0.8), (0.1, 0.5), (0.8, 0.2), (0.4, 0))

vx = [v[0] for v in vertex]
vy = [v[1] for v in vertex]

def show_graph(ax, best):
    ax.add_line(Line2D((vertex[0][0], vertex[1][0]), (vertex[0][1], vertex[1][1]
```

Далее, через цикл перебираем все маршруты, которые были получены.
Далее, при помощи класса **Line2D** прорисовываем маршруты на графе.

```python
    ax.add_line(Line2D((vertex[4][0], vertex[5][0]), (vertex[4][1], vertex[5][1]), color='#aaa'))

    startV = 0
    for i, v in enumerate(best):
        if i == 0:
            continue

        prev = startV
        v = v[:v.index(i)+1]
        for j in v:
            ax.add_line(Line2D((vertex[prev][0], vertex[j][0]), (vertex[prev][1], vertex[j][1]), color='r'))
            prev = j

    ax.plot(vx, vy, ' ob', markersize=15)
```

Далее, в исходном файле будем вызывать функция **show_ graph**. Для этого ее импортируем (из модуля **graph_show** импортируем функцию **show_ graph**).

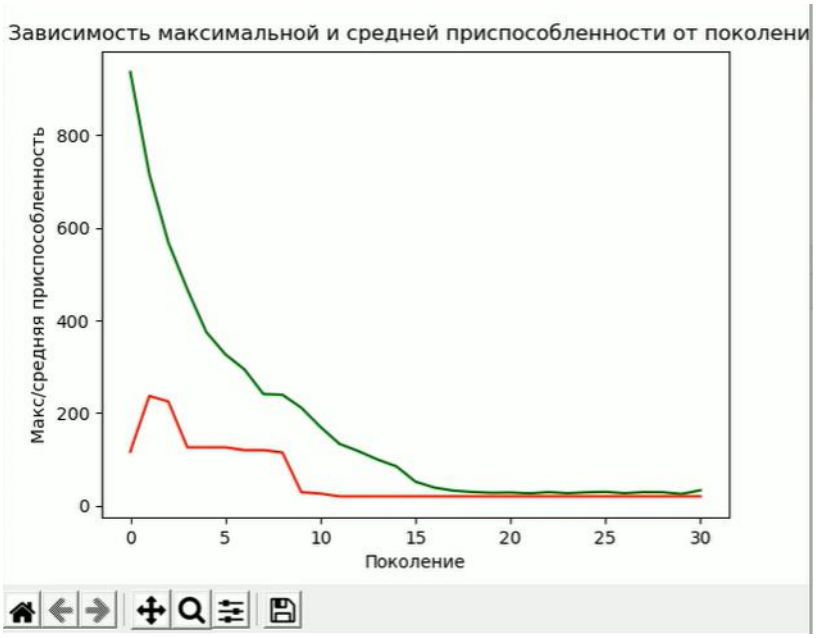
ex1.py graph_show.py

```
1  from deap import base, algorithms
2  from deap import creator
3  from deap import tools
4
5  from graph_show import show_graph
6
7  import random
8  import matplotlib.pyplot as plt
9  import numpy as np
10
11 inf = 100
```

Далее, после того, как графики выведены, добавим отображение этого графа.

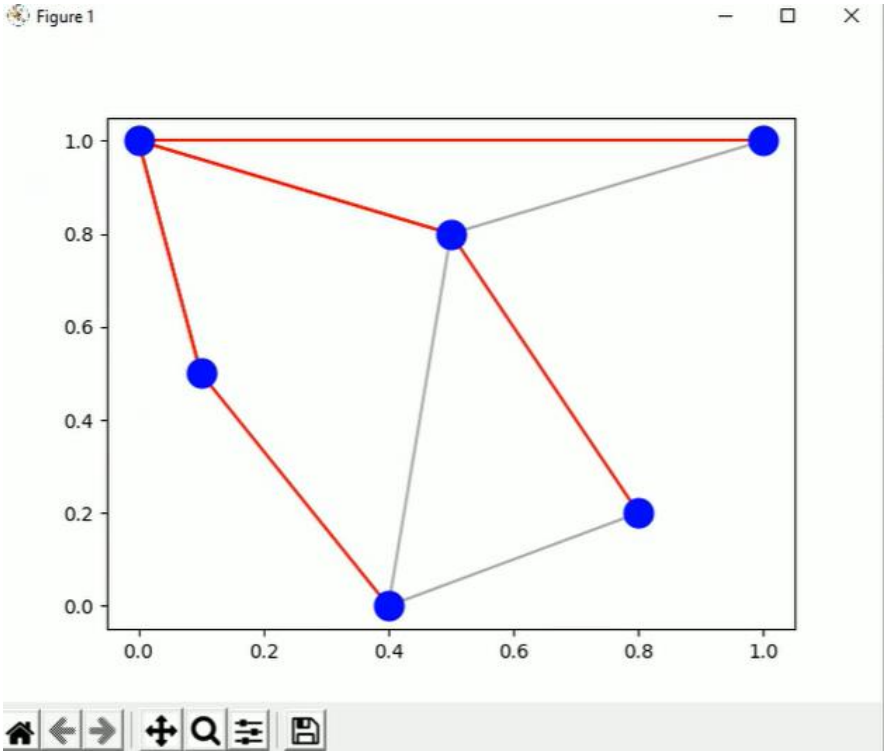
```
92  plt.plot(maxFitnessValues, color='red')
93  plt.plot(meanFitnessValues, color='green')
94  plt.xlabel('Поколение')
95  plt.ylabel('Макс/средняя приспособленность')
96  plt.title('Зависимость максимальной и средней приспособленности от поколения')
97
98  plt.show()
99
100 fig, ax = plt.subplots()
101 show_graph(ax, best)
102 plt.show()
103
```

Запустим программу. Отображается первый график.



```
[[0, 1, 4, 3, 5, 2], [1, 4, 3, 0, 5, 2], [2, 0, 5, 1, 4, 3], [3, 2, 4, 5, 1, 0], [2, 4, 1, 5, 3, 0], [3, 5, 1, 4, 0, 2]
```

Закрываем этот график и открываем следующий график, который показывает кратчайшие маршруты.



Таким образом, задача решена. График построен.

2. Поисковые задачи и комбинаторная оптимизация

Одна из основных областей применений генетических алгоритмов – поисковые задачи, которые находят широкое применение в логистике, исследовании операций, искусственном интеллекте и машинном обучении. Примерами могут служить нахождение оптимального маршрута доставки посылок, проектирование сетей транспортных авиалиний с хабами, управление портфельными инвестициями и распределение пассажиров между свободными машинами такси.

Поисковый алгоритм решает задачу путем методичного перебора состояний и переходов состояний с целью найти путь из начального состояния в конечное (или целевое). Обычно с каждым переходом состояний ассоциированы некоторые затраты (стоимость) или доходы (выигрыш), а цель поискового алгоритма – найти путь, минимизирующий затраты или максимизирующий выигрыш. Поскольку оптимальный путь – один из многих возможных, такой поиск неразрывно связан с комбинаторной оптимизацией – нахождением оптимального объекта в конечном, но очень большом множестве.

Технические требования

В этой главе нам понадобится Python 3 и следующие библиотеки:

- `deap`;
- `numpy`;
- `matplotlib`;
- `seaborn`.

Решение задачи о рюкзаке

Вспомните знакомую ситуацию – сборы перед длительной поездкой. Хочется взять с собой много всего, но место в чемодане ограничено. Мысленно вы приписываете ценность каждому предмету, но у каждого предмета есть также размер (и вес), так что все они конкурируют между собой за место в чемодане. Это одна из многих жизненных ситуаций, в которых возникает задача о рюкзаке, считающаяся самой старой и хорошо изученной задачей комбинаторной оптимизации.

Формально в задаче о рюкзаке имеются следующие компоненты:

- набор предметов, каждый из которых имеет ценность и вес;
- рюкзак (сумка, контейнер), в который можно поместить предметы ограниченного суммарного веса.

Наша цель – отобрать группу предметов максимальной суммарной ценности, так чтобы суммарный вес не превышал емкость контейнера.

В контексте алгоритмов поиска каждый набор предметов представляет *состояние*, а множество всех возможных наборов считается *пространством состояний*. Например, в задаче о рюкзаке 0-1 с n предметами размер пространства состояний равен 2^n и растет очень быстро даже при сравнительно небольших n.

В этой (классической) постановке каждый предмет либо включается ровно один раз, либо не включается вообще, поэтому иногда говорят о рюкзаке 0-1. Но возможны и другие постановки, например каждый предмет может включаться несколько раз (ограниченное или неограниченное количество) или имеется несколько рюкзаков разной емкости.

Задача о рюкзаке возникает во многих реальных процессах, где требуется принимать решения, связанные с выделением ресурсов, например: состав активов в инвестиционном портфеле, минимизация отходов при распиле, получение максимального количества баллов при решении о том, на какие вопросы отвечать в тесте с ограниченным временем.

Чтобы лучше почувствовать задачу, рассмотрим широко известный пример.

Задача о рюкзаке 0-1 с сайта Rosetta Code

На сайте Rosetta Code (`rosettacode.org`) предлагается набор задач по программированию с решениями на разных языках. Одна из них, размещенная по адресу `rosettacode.org/wiki/Knapsack_problem/0-1`, – задача о рюкзаке 0-1, в которой турист должен решить, какие предметы взять с собой в недельную поездку. Всего имеется 22 предмета, каждому из которых турист назначает некоторую ценность, отражающую его важность в путешествии.

В этой задаче максимальная емкость чемодана составляет 400. Список предметов с указанием ценности и веса приведен ниже.

Предмет	Вес	Ценность
Карта	9	150
Компас	13	35
Вода	153	200
Сэндвич	50	160
Глюкоза	15	60
Кружка	68	45
Банан	27	60
Яблоко	39	40
Сыр	23	30
Пиво	52	10
Крем от загара	11	70
Камера	32	30
Футболка	24	15
Брюки	48	10
Зонтик	73	40

Предмет	Вес	Ценность
Непромокаемые штаны	42	70
Непромокаемый плащ	43	75
Бумажник	22	80
Солнечные очки	7	20
Полотенце	18	12
Носки	4	50
Книга	30	10

Прежде чем приступать к решению, нужно обсудить важный вопрос – как мы будем представлять потенциальное решение?

Представление решения

Самый очевидный способ представить решение задачи о рюкзаке 0-1 – воспользоваться списком двоичных значений. Каждый элемент списка соответствует одному предмету. Следовательно, в задаче с сайта Rosetta Code решение будет представлено списком 22 целых чисел, принимающих значения 0 или 1. Значение 1 означает, что турист взял соответствующий предмет, а значение 0 – что оставил. В генетическом алгоритме этот список двоичных значений будет играть роль хромосомы.

Но следует помнить, что суммарный вес выбранных предметов не должен быть больше емкости рюкзака. Один из способов учесть это ограничение – отложить решение до момента оценки. В процессе вычисления оценки мы складываем веса выбранных предметов по одному, игнорируя предметы, которые привели бы к перевесу. С точки зрения генетического алгоритма, это означает, что представление индивидуума в виде хромосомы (**генотип**) может оказаться недостаточным при переходе к фактическому решению (**фенотипу**), поскольку некоторые единичные гены хромосомы игнорируются. Иногда этот переход называют *отображением генотипа в фенотип*.

В следующем разделе мы реализуем описанное представление в классе Python.

Представление задачи на Python

Для инкапсуляции данных в задаче о рюкзаке 0-1 создадим класс Python `Knapsack01Problem`. Его код находится в файле `knapsack.py` по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter04/knapsack.py.

Класс предоставляет следующие методы:

- `__init_data()`: инициализирует данные, создавая список кортежей. Каждый кортеж содержит название предмета, его вес и ценность;

- `getValue(zeroOneList)`: вычисляет суммарную ценность всех помещенных в список предметов, игнорируя те, которые привели бы к перевесу;
- `printItems(zeroOneList)`: печатает выбранные предметы, игнорируя те, которые привели бы к перевесу.

Метод `main` класса создает экземпляр класса `Knapsack01Problem`. Затем он порождает случайное решение и печатает информацию о нем. Если выполнить этот класс как автономную Python-программу, то результат будет выглядеть следующим образом:

```
Случайное решение =
[1 1 1 1 1 0 0 0 0 1 1 1 0 1 0 0 0 1 0 0 0 0]
- Добавляется карта: вес = 9, ценность = 150, вес нарастающим итогом = 9, ценность
нарастающим итогом = 150
- Добавляется компас: вес = 13, ценность = 35, вес нарастающим итогом = 22,
ценность нарастающим итогом = 185
- Добавляется вода: вес = 153, ценность = 200, вес нарастающим итогом = 175,
ценность нарастающим итогом = 385
- Добавляется сэндвич: вес = 50, ценность = 160, вес нарастающим итогом = 225,
ценность нарастающим итогом = 545
- Добавляется глюкоза: вес = 15, ценность = 60, вес нарастающим итогом = 240,
ценность нарастающим итогом = 605
- Добавляется пиво: вес = 52, ценность = 10, вес нарастающим итогом = 292,
ценность нарастающим итогом = 615
- Добавляется крем от загара: вес = 11, ценность = 70, вес нарастающим итогом = 303,
ценность нарастающим итогом = 685
- Добавляется камера: вес = 32, ценность = 30, вес нарастающим итогом = 335,
ценность нарастающим итогом = 715
- Добавляется брюки: вес = 48, ценность = 10, вес нарастающим итогом = 383,
ценность нарастающим итогом = 725
- Суммарный вес = 383, Суммарная ценность = 725
```

Заметим, что последнее вхождение 1 в случайное решение, соответствующее бумажнику, пало жертвой отображения генотипа в фенотип, о котором было сказано в предыдущем разделе. Поскольку вес этого предмета равен 22, суммарный вес превысил бы 400, поэтому он и не был включен в состав решения.

Понятно, что это случайное решение далеко от оптимальности. Попробуем найти оптимальное решение задачи, воспользовавшись генетическим алгоритмом.

Решение с помощью генетического алгоритма

Для решения задачи о рюкзаке 0-1 мы написали на Python программу, находящуюся в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter04/01-solve-knapsack.py.

Напомним, что представлением хромосомы является список целых чисел, принимающих значение 0 или 1. Поэтому задача принимает вид, похожий на задачу OneMax, решенную в предыдущей главе. Генетическому алгоритму все равно, что именно (какой фенотип) представляет хромосома – список

подлежащих упаковке предметов, коэффициент булева уравнения или просто какое-то двоичное число, – его интересует только сама хромосома (генотип) и ее приспособленность. Отображение хромосомы на представляемое ей решение производится функцией приспособленности, реализованной вне генетического алгоритма. В нашем случае это отображение хромосомы и вычисление приспособленности реализованы методом `getValue()` класса `Knapsack01Problem`.

Это означает, что мы можем воспользоваться той же реализацией генетического алгоритма, что и для задачи OneMax, внеся лишь несколько модификаций.

Ниже описаны основные моменты нашего решения.

1. Сначала создаем экземпляр решаемой задачи о рюкзаке:

```
knapsack = knapsack.Knapsack01Problem()
```

2. Затем просим генетический алгоритм вызвать метод `getValue()` этого экземпляра для вычисления приспособленности:

```
def knapsackValue(individual):
    return knapsack.getValue(individual),

toolbox.register("evaluate", knapsackValue)
```

3. Используемые генетические операторы совместимы с представлением хромосомы списком двоичных значений:

```
toolbox.register("select", tools.selTournament, tournsize=3)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/len(knapsack))
```

4. По завершении генетического алгоритма вызываем метод `printItems()` для печати найденного лучшего решения:

```
best = hof.items[0]

print("-- Предметы в рюкзаке = ")
knapsack.printItems(best)
```

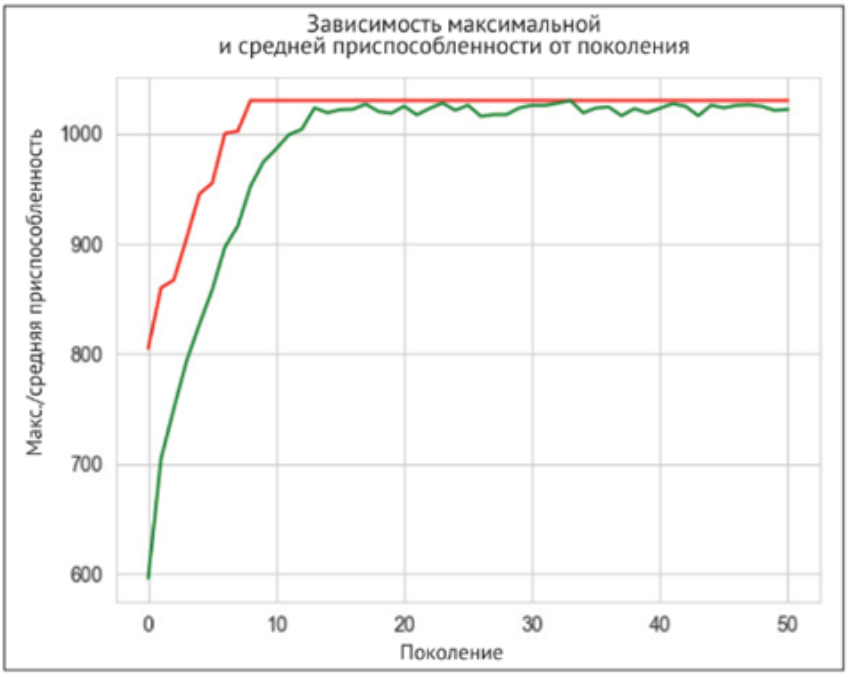
5. Можно также изменить некоторые параметры генетического алгоритма. Поскольку в этой задаче длина двоичной строки равна 22, можно надеяться, что она проще задачи OneMax со строкой длины 100, поэтому попробуем уменьшить размер популяции и максимальное количество поколений.

Прогон алгоритма с 50 поколениями и популяцией размера 50 дает следующий результат:

```
-- Лучший индивидуум = [1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0,
0, 0, 0, 1, 1, 1, 1, 0, 1, 1]
-- Лучшая приспособленность = 1030.0
-- Предметы в рюкзаке =
- Добавляется карта: вес = 9, ценность = 150, вес нарастающим итогом  = 9,
ценность нарастающим итогом = 150
- Добавляется  компас: вес = 13, ценность = 35, вес нарастающим итогом  =
22, ценность нарастающим итогом = 185
- Добавляется  вода: вес = 153, ценность = 200, вес нарастающим итогом  =
175, ценность нарастающим итогом = 385
- Добавляется  сэндвич: вес = 50, ценность = 160, вес нарастающим итогом
= 225, ценность нарастающим итогом = 545
- Добавляется  глюкоза: вес = 15, ценность = 60, вес нарастающим итогом  =
240, ценность нарастающим итогом = 605
- Добавляется  банан: вес = 27, ценность = 60, вес нарастающим итогом  =
267, ценность нарастающим итогом = 665
- Добавляется  крем от загара: вес = 11, ценность = 70, суммарный
вес = 278, ценность нарастающим итогом = 735
- Добавляется  непромокаемые штаны: вес = 42, ценность = 70,
вес нарастающим итогом  = 320, ценность нарастающим итогом = 805
- Добавляется  непромокаемый плащ: вес = 43, ценность = 75,
вес нарастающим итогом  = 363, ценность нарастающим итогом = 880
- Добавляется  бумажник: вес = 22, ценность = 80, вес нарастающим итогом
= 385, ценность нарастающим итогом = 960
- Добавляется  солнечные очки: вес = 7, ценность = 20, вес нарастающим итогом
= 392, ценность нарастающим итогом = 980
- Добавляется  носки: вес = 4, ценность = 50, вес нарастающим итогом  =
396, ценность нарастающим итогом = 1030
- Суммарный вес = 396, суммарная ценность = 1030
```

Известно, что 1030 – действительно оптимальное решение этой задачи. И снова мы видим, что последнее вхождение 1 в хромосому лучшего индивидуума, представляющее книгу, выпало из отображения на фактическое решение, чтобы суммарный вес не вышел за пределы 400.

Из графиков зависимости максимальной и средней приспособленности от номера поколения видно, что лучшее решение было найдено меньше, чем за 10 поколений.



Статистика программы для решения задачи о рюкзаке 0-1

3. Задачи с ограничениями.

Технические требования

В этой главе нам понадобится Python 3 и следующие библиотеки:

- `deap`;
- `numpy`;
- `matplotlib`;
- `seaborn`;
- `networkx` (о ней будет рассказано здесь же).

Соблюдение ограничений в поисковых задачах

В предыдущей главе мы рассмотрели решение задач поиска, заключающееся в методичном оценивании состояний и переходов между состояниями. С каждым переходом состояний ассоциированы затраты или доходы, а цель заключается в том, чтобы минимизировать затраты или максимизировать доходы. Задачи с ограничениями – разновидность поисковых задач, в которых состояния должны удовлетворять ряду ограничений. Если нам удастся выразить ограничения в виде затрат, а затем минимизировать затраты, то задача с ограничениями сведется к общей задаче поиска.

Как и задачи комбинаторной оптимизации, задачи с ограничениями имеют важные применения в таких областях, как искусственный интеллект, исследование операций и распознавание образов. Хорошее понимание природы таких задач может помочь при решении разнообразных, на первый взгляд, несвязанных проблем. Поскольку задачи с ограничениями бывают очень трудны, открываются широкие перспективы для применения генетических алгоритмов.

Решение задачи о составлении графика дежурств медсестер

Пусть требуется составить недельный график дежурств медсестер в отделении больницы. В сутках три смены – утренняя, дневная и вечерняя, и в каждой смене должно быть не менее одной из восьми работающих в отделении медсестер. Если вам кажется, что это просто, примите во внимание перечень действующих в больнице правил:

- сестре не разрешается работать две смены подряд;
- сестре не разрешается работать более пяти смен в неделю;
- количество сестер в смене должно удовлетворять дополнительным ограничениям:
  - в утренней смене: 2–3 сестры;
  - в дневной смене: 2–4 сестры;
  - в ночной смене: 1–2 сестры.

Кроме того, у каждой сестры могут быть пожелания. Например, одна сестра предпочитает только утренние смены, другая не хочет работать днем и т. д.

Эта задача о составлении графика дежурств медсестер (СГДМ) имеет много вариантов. Например, у сестер могут быть различные специальности, можно разрешить сверхурочную работу, даже сами смены могут быть разными – скажем, 8- и 12-часовыми.

Теперь вы, наверное, согласитесь, что лучше поручить составление графика программе. И почему бы не воспользоваться нашими знаниями о генетических алгоритмах? Как обычно, начнем с представления решения.

Представление решения

Мы решили представлять график в виде двоичного списка (или массива), поскольку интуитивно понятно, как его интерпретировать, а с точки зрения генетических алгоритмов такое представление выглядит естественно.

Каждой сестре соответствует двоичная строка, представляющая 21 смену в неделю. Значение 1 означает, что сестра должна выйти в эту смену. Рассмотрим, например, следующий список:

```
(0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0)
```

Его можно разбить на группы по три значения, представляющие смены, в которые сестра выходит на работу в каждый день недели:

Воскресенье	Понедельник	Вторник	Среда	Четверг	Пятница	Суббота
(0,1,0)	(1,0,1)	(0,1,1)	(0,0,0)	(0,0,1)	(1,0,0)	(0,1,0)
дневная	утренняя и ночная	дневная и ночная	не выходит	ночная	утренняя	дневная

Графики дежурств всех сестер можно конкатенировать, создав один длинный двоичный список, представляющий полное решение.

При оценке решения этот длинный список можно разбить на графики отдельных медсестер и проверять нарушения ограничений. Так, в примере выше имеется два случая, когда идут подряд две единицы (дневная и ночная смены, а позже – ночная и утренняя). Количество смен в неделю для данной сестры можно вычислить, подсчитав количество единиц, – получится 8 смен. Можно также проверить, удовлетворяются ли пожелания медсестер, для чего нужно сравнить назначение сестры на смены с ее пожеланиями.

Наконец, чтобы проверить ограничения на количество сестер в смене, можно просуммировать недельные графики всех сестер и посмотреть, есть ли смены, в которых сестер больше максимально допустимого или меньше минимально допустимого предела.

Но прежде чем переходить к реализации, нужно обсудить различие между жесткими и мягкими ограничениями.

Жесткие и мягкие ограничения

При составлении графика дежурств медсестер следует помнить, что некоторые больничные правила нарушать нельзя. Если график содержит хотя бы одно нарушение этих правил, то он считается недопустимым. В общем случае такие ограничения называются **жесткими**.

С другой стороны, пожелания медсестер можно считать **мягкими ограничениями**. Мы хотели бы их удовлетворить, и решение, содержащее мало таких нарушений или не содержащее их вовсе, считается лучше, чем решение с большим числом нарушений. Тем не менее нарушение мягких ограничений не делает решение недопустимым.

С мягкими ограничениями мы обращаемся так же, как в любой задаче оптимизации, т. е. пытаемся минимизировать количество их нарушений, но вот как быть с жесткими ограничениями? Есть несколько стратегий.

- Найти такое представление (кодировку) решения, при котором **исключается сама возможность** нарушить жесткое ограничение.

- При оценивании решений **отбрасывать** те, что нарушают хотя бы одно жесткое ограничение. Недостаток такого подхода – в том, что информация, содержащаяся в этих решениях, теряется, а она может оказаться ценной. В результате процесс оптимизации сильно замедляется.
- При оценивании решений **исправлять** те, что нарушают жесткие ограничения. Иными словами, найти такой способ модификации решения, который устранял бы нарушения. Для большинства задач найти подобную процедуру исправления трудно или невозможно, к тому же сам процесс исправления может привести к значительной потере информации.
- При оценивании решений штрафовать те, что нарушают жесткие ограничения. Это уменьшает оценку решения и делает его менее желательным, но не исключает полностью, поэтому содержащаяся в нем информация не теряется. При таком подходе жесткое ограничение рассматривается как мягкое, но с более высоким штрафом. Проблема в том, как выбрать подходящую величину штрафа. Если штраф слишком велик, то решения, по сути дела, исключаются, а если слишком мал, то недопустимое решение может оказаться оптимальным.

Мы остановимся на четвертом подходе – будем штрафовать нарушения жестких ограничений сильнее, чем мягких. Для этого мы напишем функцию стоимости, в которой цена нарушения жесткого ограничения больше, чем мягкого. Затем полная стоимость будет использоваться как функция приспособленности, подлежащая минимизации. Все это реализовано в представлении задачи, которое мы обсудим в следующем подразделе.

Представление задачи на Python

Для инкапсуляции задачи о составлении графика дежурств мы создали Python-класс `NurseSchedulingProblem`. Он находится в файле https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter05/nurses.py.

Конструктор класса принимает параметр `hardConstraintPenalty` – коэффициент штрафа за нарушение жесткого ограничения и инициализирует различные параметры задачи.

```
# список медсестер
self.nurses = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']

# пожелания медсестер к сменам - утренняя, дневная, ночная
self.shiftPreference = [[1, 0, 0], [1, 1, 0], [0, 1, 1], [0, 1, 0], [0, 0, 1],
[1, 1, 1], [0, 1, 1], [1, 1, 1]]
# минимальное и максимальное количество медсестер в сменах - утренней, дневной,
# ночной
```

```
self.shiftMin = [2, 2, 1]
self.shiftMax = [3, 4, 2]

# максимальное количество смен, которые разрешено отработать медсестре в неделю
self.maxShiftsPerWeek = 5
```

В классе имеется также метод для преобразования заданного графика в словарь, содержащий графики для каждой медсестры:

- `getNurseShifts(schedule)`

Следующие методы используются для подсчета нарушений разных типов:

- `countConsecutiveShiftViolations(nurseShiftsDict)`
- `countShiftsPerWeekViolations(nurseShiftsDict)`
- `countNursesPerShiftViolations(nurseShiftsDict)`
- `countShiftPreferenceViolations(nurseShiftsDict)`

Дополнительно класс предоставляет следующие открытые методы:

- `getCost(schedule)`: вычисляет полную стоимость различных нарушений графика. В этом методе используется переменная `hardConstraintPenalty`;
- `printScheduleInfo(schedule)`: печатает график и сведения о нарушениях.

Метод `main` вызывает методы класса: создает экземпляр задачи и проверяет случайно выбранное решение. Если `hardConstraintPenalty = 10`, то результат мог бы выглядеть так:

```
Случайное решение =
[1 0 0 0 0 1 1 1 0 0 0 1 1 0 0 0 1 0 1 0 1 1 0 1 0 1 1 1 0 1 0 1 0 1 1 1 1
 0 0 1 0 1 0 0 1 0 1 1 0 1 1 0 1 1 0 1 1 1 1 0 1 0 1 0 1 1 0 1 0 1 1 1
 1 0 1 0 0 0 1 1 0 1 1 1 1 0 1 1 0 1 1 1 1 1 0 1 0 0 1 1 0 1 1 1 0 0 0 0 0
 0 1 0 1 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 1 1 0 0 0 0 1 1 0 1 1 0
 0 1 0 1 1 1 0 0 0 0 0 0 0 1 1 1 0 0 1 1]

Графики медсестер:
A : [1 0 0 0 0 1 1 1 0 0 0 1 1 0 0 0 1 0 1 0 1]
B : [1 0 1 0 1 1 1 0 1 0 1 0 1 1 1 1 0 0 1 0 1]
C : [0 0 1 0 1 1 0 1 1 0 1 1 0 1 1 1 1 0 1 0 1]
D : [0 1 0 1 1 0 1 0 1 1 1 1 0 1 0 0 0 1 1 0 1]
E : [1 1 1 0 1 1 0 1 1 1 1 1 1 0 1 0 0 1 1 0 1 1]
F : [1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 1 0 0 0 0 0]
G : [0 0 0 0 1 1 0 0 1 1 1 1 0 0 0 0 1 1 0 1 1]
H : [0 0 1 0 1 1 1 0 0 0 0 0 0 0 1 1 1 0 0 1 1]
нарушений ограничения на две смены подряд = 40

смен в неделю = [9, 13, 13, 12, 15, 5, 10, 9]
нарушений ограничения на число смен в неделю = 46

сестер в смене = [4, 2, 4, 1, 6, 6, 4, 4, 5, 4, 5, 5, 2, 4, 4, 4, 5, 3, 4, 3, 7]
нарушений ограничения на число сестер в смене = 30

нарушений пожеланий сестер = 30

Общая стоимость = 1190
```

Как видим, случайно сгенерированное решение, скорее всего, будет содержать много нарушений, а значит, его стоимость будет высока. В следующем подразделе мы попытаемся минимизировать стоимость и исключить все нарушения жестких ограничений.

Решение на основе генетического алгоритма

Для решения задачи о составлении графика дежурств мы написали на Python программу, которая находится в файле https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter05/02-solve-nurses.py.

Поскольку решение представлено списком (или массивом) двоичных значений, мы можем воспользоваться тем же подходом, который успешно применяли к решению других задач, например задачи о рюкзаке в главе 4.

Ниже описаны основные шаги решения.

1. Сначала создаем экземпляр класса `NurseSchedulingProblem` со значением `hardConstraintPenalty`, равным константе `HARD_CONSTRAINT_PENALTY`:

```
nsp = nurses.NurseSchedulingProblem(HARD_CONSTRAINT_PENALTY)
```

2. Поскольку мы стремимся минимизировать стоимость, определяем единственную цель – минимизирующую стратегию приспособления:

```
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
```

3. Поскольку решение представлено списком, содержащим значения 0 и 1, воспользуемся следующими элементами инструментария для создания начальной популяции:

```
toolbox.register("zeroOrOne", random.randint, 0, 1)
toolbox.register("individualCreator", tools.initRepeat,
creator.Individual, toolbox.zeroOrOne, len(nsp))
toolbox.register("populationCreator", tools.initRepeat, list,
toolbox.individualCreator)
```

4. Функция приспособленности вычисляет стоимость различных нарушений в графике, представленном одним решением:

```
def getCost(individual):
    return nsp.getCost(individual),

toolbox.register("evaluate", getCost)
```

5. В качестве оператора отбора берем турнир размера 2, будем также использовать двухточечный оператор скрещивания и мутацию инвертированием бита, поскольку они удобны при работе с двоичными списками:

```
toolbox.register("select", tools.selTournament, tournsize=2)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/len(nsp))
```

6. Продолжаем применять элитизм, а конкретно копируем без изменения лучших индивидуумов из зала славы в следующее поколение:

```
population, logbook = elitism.eaSimpleWithElitism(population,
toolbox, cxpb=P_CROSSOVER, mutpb=P_MUTATION,
ngen=MAX_GENERATIONS, stats=stats, halloffame=hof, verbose=True)
```

7. По завершении алгоритма распечатываем сведения о лучших найденных решениях:

```
nsp.printScheduleInfo(best)
```

Прежде чем запускать программу, зададим параметры алгоритма:

```
POPULATION_SIZE = 300
P_CROSSOVER = 0.9
P_MUTATION = 0.1
MAX_GENERATIONS = 200
HALL_OF_FAME_SIZE = 30
```

Кроме того, зададим штраф за нарушение жестких ограничений равным 1, уравняв тем самым стоимость нарушения жестких и мягких ограничений:

```
HARD_CONSTRAINT_PENALTY = 1
```

Программа, запущенная с такими параметрами, печатает следующую информацию:

```
-- Лучшая приспособленность = 3.0
-- График =
Графики медсестер:
A : [0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0]
B : [1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0]
C : [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
D : [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1]
E : [0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0]
F : [0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0]
G : [0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1]
H : [1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0]
нарушений ограничения на две смены подряд = 0

смен в неделю = [5, 6, 2, 5, 4, 5, 5, 5]
нарушений ограничения на число смен в неделю = 1

сестер в смене = [2, 2, 1, 2, 2, 1, 2, 2, 1, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 1]
нарушений ограничения на число сестер в смене = 0

нарушений пожеланий сестер = 2
```

Может показаться, что это неплохо, потому что имеется всего три нарушения ограничений. Но одно из них – несоблюдение количества смен в неделю: сестре B запланировано шесть смен, тогда как разрешено не больше пяти. Этого достаточно, чтобы решение было признано недопустимым.

Попробуем избавиться от таких нарушений, задав штраф за нарушение жесткого ограничения равным 10:

```
HARD_CONSTRAINT_PENALTY = 10
```

Теперь результат выглядит так:

```
-- Лучшая приспособленность = 3.0
-- График =
Графики медсестер:
A : [0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
B : [1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0]
C : [0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1]
D : [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0]
E : [0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
F : [0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0]
G : [0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0]
H : [1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0]
нарушений ограничения на две смены подряд = 0

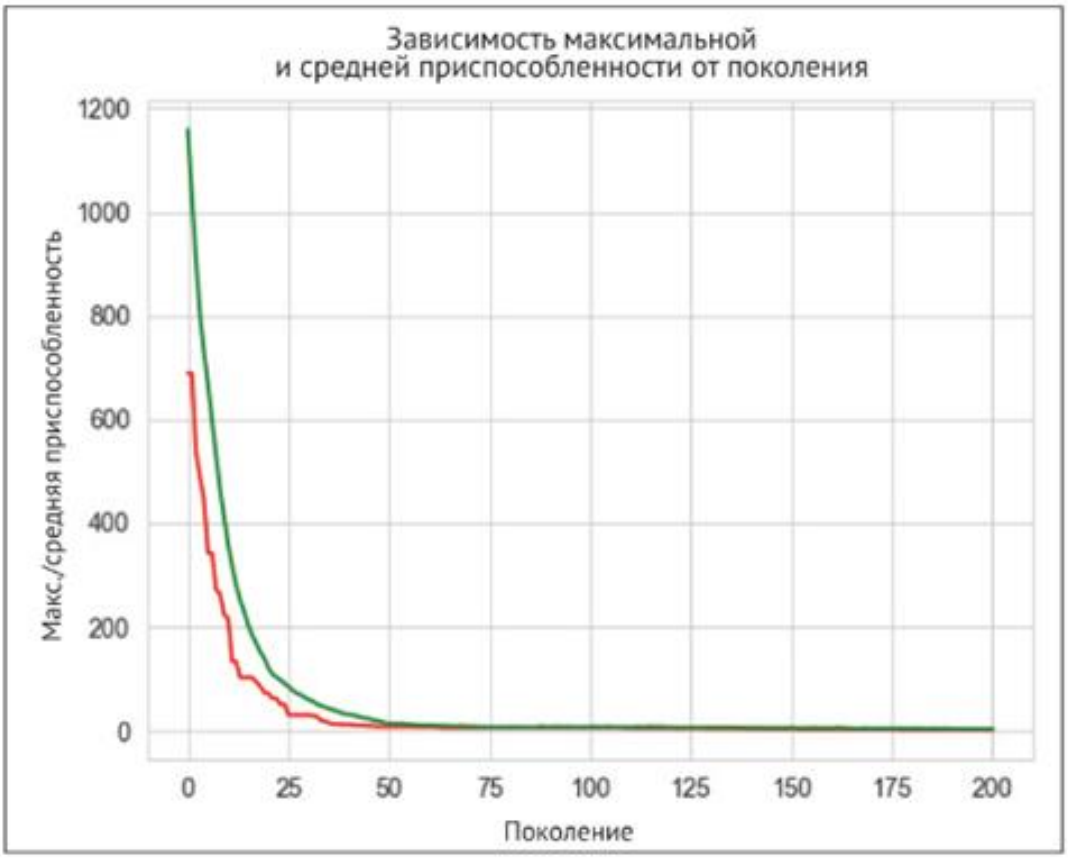
смен в неделю = [4, 5, 5, 5, 3, 5, 5, 5]
нарушений ограничения на число смен в неделю = 0

сестер в смене = [2, 2, 1, 2, 2, 1, 2, 2, 2, 2, 2, 1, 2, 2, 1, 2, 2, 2, 2, 2, 1]
нарушений ограничения на число сестер в смене = 0

нарушений пожеланий сестер = 3
```

Нарушений снова три, но на этот раз нарушены только мягкие ограничения, поэтому решение допустимо.

На рисунке ниже изображены графики зависимости минимальной и средней приспособленности от номера поколения. Мы видим, что за первые 40–50 поколений алгоритм сумел исключить все нарушения жестких ограничений, после чего были лишь небольшие улучшения, связанные с исключением очередного нарушения мягких ограничений.



Статистика программы для задачи
о составлении графика дежурств медсестер

В нашей задаче оказалось достаточно сделать штраф за нарушение жестких ограничений в 10 раз больше. В других задачах может понадобиться более высокое значение. Попробуйте поэкспериментировать с другими параметрами, а также с определениями генетических операторов.

Задания для самостоятельной работы

1. Изучить **задачу поиска минимальных маршрутов в графе**. Составить граф, состоящий из 3 или 4 ярусов (13...19 вершин). Указать максимальное количество связей и задать вес. На основе этих данных решить задачу с использованием пакета DEAP.

2. Изучить **поисковые задачи и комбинаторную оптимизацию**. Составить «свой рюкзак» (15…30 предметов). На основе этих данных решить задачу с использованием пакета DEAP.

3. Изучить **задачи с ограничениями**. Составить «свое расписание дежурств». На основе этих данных решить задачу с использованием пакета DEAP.